

Key Recovery from Side-Channel Power Analysis Attacks on Non-SIMD HQC Decryption

Nathan Maillet^{1,3} , Cyrius Nugier² , Vincent Migliore³ , and
Jean-Christophe Deneuville² 

¹ EDF R&D, France

² Fédération ENAC ISAE-SUPAERO ONERA, Université de Toulouse, France

³ LAAS-CNRS, Université de Toulouse, INSA-Toulouse, France

<firstname>.<lastname>@<edf/enac/laas>.fr

Abstract. HQC is a code-based cryptosystem that has recently been announced for standardization after the fourth round of the NIST post-quantum cryptography standardization process. During this process, the NIST specifically required submitters to provide two kinds of implementation: a reference one, meant to serve lisibility and compliance with the specifications; and an optimized one, aimed at showing the performance of the scheme alongside other desirable properties such as resilience against implementation misuse or side-channel analysis. While most side-channel attacks regarding PQC candidates running in this process were mounted over reference implementations, very few consider the optimized, allegedly side-channel resistant (at least, constant-time), implementations. Unfortunately, HQC optimized version only targets x86-64 with Single Instruction Multiple Data (SIMD) support, which reduces the code portability, especially for non-generalist computers. In this work, we present two power side-channel attacks on the optimized HQC implementation with just the SIMD support deactivated. We show that the power leaks enough information to recover the private key, assuming the adversary can ask the target to replay a legitimate decryption with the same inputs. Under this assumption, we first present a key-recovery attack targeting standard Instruction Set Architectures (ARM T32, RISC-V, x86-64) and compiler optimization levels. It is based on the well known Hamming Distance model of power consumption leakage, and exposes the key from a single oracle call. During execution on a real target, we show that a different leakage, stemming from the micro-architecture, simplifies the recovery of the private key. This more direct second attack, succeeds with a 99% chance from 83 executions of the same legitimate decryption. While the weakness leveraged in this work seems quite devastating, we discuss simple yet effective and efficient countermeasures to prevent such a key-recovery.

1 Introduction

The first code-based cryptosystem dates back to 1978, with the seminal work of Robert J. McEliece [28]. He suggested to use binary Goppa codes, for which a

disguised version can be made public. A plaintext is encrypted by first encoding it, then adding a small amount of noise (below the capacity of the Goppa code) on top of it. Only the knowledge of the associated decoding algorithm allows to decrypt. While McEliece’s proposal was disregarded for decades in favor of the so-called RSA cryptosystem [41], three orders of magnitude more compact, it has regained interest since the introduction of quantum attacks [48]. McEliece’s scheme relies upon two security assumptions: the syndrome decoding problem, and the indistinguishability of binary Goppa codes among random linear codes. While the former has been proved NP-complete [7], the hardness of the latter is more ambiguous. To date, binary Goppa codes with rates close to the ones suggested by McEliece seem to withstand distinguishers^{1,2}, but changing the family of codes to a more structured one has frequently led to key-recovery attacks.

In 2003, Alekhnovich proposed a public key encryption scheme whose security solely relies on the hardness of decoding random linear codes [4]. Although the resulting scheme provides strong security guarantees, the resulting parameters are completely impracticable. Nevertheless, it has inspired several modern lattice-based and code-based constructions, such as the Hamming Quasi-Cyclic cryptosystem [3].

Following the Alekhnovich blueprint, HQC was originally proposed as a public key encryption scheme, differing from McEliece in two ways [3]. First, it uses two codes instead of one, the first one being a random quasi-cyclic code, which allows to rely on the hardness of the (decisional) syndrome decoding problem (on random quasi-cyclic codes).³ The second code is a public one, used for encryption. The fact that this code is public constitutes the second main difference with McEliece: the decoding algorithm is hence public. The main difference here is to add more errors to the encoded plaintext than the correction capability of the code. Only the knowledge of the private key allows to remove sufficiently many errors to fall back under this correction capability threshold (with overwhelming probability). Making this code public hence eliminates every distinguishing attacks (e.g. code squaring [10]) affecting improvements over McEliece based on structured codes, and potentially McEliece’s scheme itself. On the downside, decryption failures need to be handled carefully.

1.1 Reaction attacks and side-channel

Indeed, it was shown that an improperly handled decoding failure, leading to a decryption failure, can be leveraged to recover the private key [17]. More dramatically, this kind of attack tends to be the most efficient strategy when the decryption failure rate (DFR) is not negligible in the security parameter. This induces a strong constraint in the design of post-quantum cryptosystems.

¹ this does not hold for high rate binary Goppa codes [11].

² a recent Eurocrypt paper improves distinguishing binary Goppa codes in the regime of parameters relevant to cryptography below an exponential complexity [39].

³ the regime of parameters considered within HQC differs from the NP-completeness result of [7].

Fortunately, this DFR can sometimes be easy to prove negligible, as it is the case for HQC. This constraint however have an impact on the efficiency.

The framework in [17] establishes a connection between the key distribution (spectrum) and the observed decryption failure. Exploiting such a decryption failure to recover the private key *does not* require side-channel analysis capability, only an adaptive adversary.

Protecting against reaction attacks unfortunately does not provide guarantees for side-channel resistance. It has already been shown [14,44,43] that the ability to observe the decoding algorithm can yield security losses.

The aforementioned side-channel attacks (SCA) can exploit well-known implementation mistakes such as conditional branching on secret data, secret-dependent memory accesses, or physical measurements such as power consumption, etc. The targeted cryptosystem is rarely claimed “side-channel resistant / protected” or even constant-time.

1.2 NIST PQC initiative and relative implementations

The National Institute of Standards and Technology (NIST) launched in 2017 a call for standardizing post-quantum primitives [34]. This initiative drew a lot of attention for designs, with 69 proposals considered for the first round (with a high representation of lattice and code based submissions). It also raised interest in cryptanalysis, including side-channel ones.

Within this call, to be considered proper and complete, a submission was expected to provide specifications, an ANSI C 99 reference implementation,⁴ and an optimized implementation.

It is important to highlight that the reference implementation, as per NIST recommendation [34, Sec. 2.C.1, p. 8], was meant:

“to promote understanding of how the submitted algorithm may be implemented. Since this implementation is intended for reference purposes, clarity in the implementation code is more important than the efficiency of the code.”

On the other hand, optimized implementations would serve efficiency purposes, and possibly demonstrate side-channel resilience [34, Sec. 4.A.6, p. 19]:

“Another case where security and performance interact is resistance to side-channel attacks. Schemes that can be made resistant to side-channel attack at minimal cost are more desirable than those whose performance is severely hampered by any attempt to resist side-channel attacks. We further note that optimized implementations that address side-channel attacks (e.g., constant-time implementations) are more meaningful than those which do not.”

⁴ NIST accepted dependencies with external C++ libraries such as NTL [49], yielding C++ implementations instead of regular ANSI C 99 ones.

From the above quotes, it is hence clear that reference implementations are *not*, unless explicitly claimed by the submitters, side-channel protected, or even constant-time.

This fact (that reference implementations are not supposed to demonstrate side-channel resilience such as running in constant time) does not seem to be widely known within the cryptographic community, and a lot of side-channel attacks targeting submitted reference implementations has emerged. Regarding the reference implementation of the NIST submission HQC [1], this includes power based attacks [43], cache timing attacks [20] and electromagnetic side-channel attacks [14].

While the results claimed within these papers hold for the reference implementation, this is generally not true if one tries to straightforwardly carry the attack over the optimized implementation directly.

1.3 The HQC cryptosystem and its associated decoding

As mentioned before, HQC uses two codes, the first one being a random quasi-cyclic one, hence not efficiently decodable. The choice for the second code, used to encode the plaintext, is open: any code can be used as long as it provides a negligible DFR in the security parameter. Relevant codes are usually binary, asymptotically good (in terms of relative distance), with an explicit construction and efficient encoding and decoding algorithms. Concatenated Reed-Muller and Reed-Solomon (RMRS) codes therefore seem to be a natural choice, as proposed in the NIST version [1]. The error added to the encoded plaintext also has a specific shape: it is constructed using the public key.

To decode such an HQC-RMRS ciphertext, one uses its private key to remove many errors before applying the decoding algorithm. RMRS decoding works as follows: the “cleaned” ciphertext (obtained after removing errors using the private key) is decoded by blocks with a duplicated first-order dimension 8 Reed-Muller code which, thanks to the high relative distance, removes most errors with high probability. To remove the remaining errors, the output then goes through the decoder of a shortened Reed-Solomon code, which has the perfect balance between rate and distance to ensure that a decoding failure can happen only with probability less than $2^{-\lambda}$, λ being the security parameter (at least 128 bits).

Although the decoding algorithm is publicly known, its implementation represents an attack surface that could be exploited, for instance by SCA. As the NIST standardization process ended for Key Encapsulation Mechanism (KEM) algorithms, providing side-channel resistant implementations is of utmost importance.

1.4 Our contributions

This paper focuses on the *optimized* implementation of HQC.⁵ However, this version has been proposed for x86-64 processors with (at least) AVX2 support

⁵ https://pqc-hqc.org/doc/hqc-optimized-implementation_2024-10-30.zip

and aggressive compiler optimizations only. Consequently, carrying this version over a wider range of target systems while keeping side-channel security guarantees requires careful attention, especially for non-SIMD processors or broader compilation trade-offs.

This paper contains two main contributions in the form of two side-channel attacks applicable to the optimized implementation of the HQC Round 4 submission, and targeting the `expand_and_sum` function in Reed-Muller decoding with SIMD operations deactivated. This implementation could be used in different scenarios, for example in connected objects and Internet of Things systems where SIMD modules are not systematically integrated in hardware, but not exclusively: for instance, SIMD operations are not always the best solution for a specific target equipped with SIMD module (see Table 1). In that sense, the vulnerabilities may be found in several software depending on the compiler behavior and optimisation flags. Of particular note, the `expand_and_sum` without SIMD is carried out in constant time, which meets the requirements of an optimized version.

First, we provide a generic attack on the HQC Reed-Muller decoding applicable to ARM T32, RISC-V and x86-64 Instruction Set Architectures (ISA). It applies to both the Public Key Encryption (PKE) and KEM versions. It shows that the power consumption during the start of the decoding can be exploited to retrieve the value of the ciphertext after noise removal. This information then allows the attacker to retrieve the secret key. The retrieval of the key only requires one trace with a side-channel oracle having an accuracy of 100%.

Second, we provide an attack on a target equipped with an ARM Cortex-M4 implementing one of such ISA, with power consumption measures done with a ChipWhisperer device. This practical application shows that, in comparison to the first attack, information leakage from the micro-architecture simplifies the recovery algorithm. The experiments show that, in real conditions, 83 traces are enough to ensure a key recovery with 99% for any HQC version.

Overall, this paper shows that protecting the HQC decrypt implementation from leaks of the Reed-Muller decoding input and output is crucial for its security. We propose three ways to avoid the leaks exploited in the paper: redesigning the decoding into a minimum distance decoding from the codeword list, adding noise to the ciphertext before decoding to protect the input, and masking the ciphertext with an encoded random word.

1.5 Organization of the paper

The rest of the paper is structured as follows: Section 2 introduces the mathematical background and usual notation used in the HQC cryptosystem. Section 3 presents existing side-channel attacks on HQC, Section 4 presents the attack process and how the attacker retrieves the keys from side-channel information. Section 5 presents the instruction-level attack, Section 6 presents the micro-architectural attack with a ChipWhisperer power consumption measurements, and, finally, Section 7 presents countermeasures that can be used against our attacks and the ones presented in Section 3.

Version	Cycle count (mean)	Cycle count (standard deviation)
Optimized (-03)	471	72
SIMD -02	550	239
No SIMD	312	12
Reference	669	58

Table 1: `expand_and_sum` performance comparison. Tested on an 11th Gen Intel(R) Core(TM) i7-11850H @ 2.50GHz, bare metal through an Extensible Firmware Interface (EFI) Application, maskable interrupts disabled; and external interrupts disabled with Task-Priority Register in the control register CR8 set to 0xf. The methodology followed is detailed in Appendix A.

2 Background

2.1 HQC

This subsection covers the basic description and notations linked to the HQC cryptosystem. In this paper, vectors (resp. matrices) are denoted in lowercase (resp. uppercase) bold font. A binary vector $\mathbf{v} \in \mathbb{F}_2^n$ can be interchangeably considered as a vector or a polynomial in the ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$. Its k -th coordinate is noted v_k . Its Hamming weight (the number of non-zero coordinates) is denoted $\omega(\mathbf{v})$, and the set of $\mathbb{F}_2^n$ vectors with weight w is denoted $\mathcal{S}_w^n(\mathbb{F}_2)$:

$$\mathcal{S}_w^n(\mathbb{F}_2) = \{\mathbf{v} \in \mathbb{F}_2^n \mid \omega(\mathbf{v}) = w\}$$

Given two vectors $\mathbf{u}$ and $\mathbf{v}$, their product $\mathbf{w} = \mathbf{u} \cdot \mathbf{v}$ in $\mathcal{R}$ is defined as:

$$\mathbf{w} = (w_0, \dots, w_{n-1}), \text{ where } w_k = \sum_{i+j=k \pmod n} u_i v_j$$

This product can also be computed as the product of the vector $\mathbf{u}$ and a circulant matrix associated to $\mathbf{v}$, denoted $\mathbf{rot}(\mathbf{v})$ and defined as follows:

$$\mathbf{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{n-1} & \cdots & v_1 \\ v_1 & v_0 & \cdots & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}$$

The product of any two elements $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$ is equivalently defined with the $\mathbf{rot}(\cdot)$ operator as:

$$\mathbf{w} = \mathbf{u} \cdot \mathbf{v} = \mathbf{u} \times \mathbf{rot}(\mathbf{v})^\top = (\mathbf{rot}(\mathbf{u}) \times \mathbf{v}^\top)^\top = \mathbf{v} \times \mathbf{rot}(\mathbf{u})^\top = \mathbf{v} \cdot \mathbf{u}.$$

We now recall the HQC scheme in Figure 1. In its first version, the code used was a tensor code between a BCH code and a repetition code [3]. As per round

2 (the 2020/04/21 update of HQC), it is a duplicated first-order Reed-Muller code concatenated with a shortened Reed-Solomon code [29].

The choice of the code has no impact on security, but it does have an impact on performance. First, the dimension of the code k must be larger than the security parameter λ to have a large enough message space (ideally to be able to encapsulate a λ -bits symmetric key for λ bits of security). Second, the length n of the ambient space $\mathcal{R}$ must be taken as a primitive prime larger than the code length, to avoid structural attacks [12,16,26].⁶ This means that 2 is a primitive n -th root of unity, or equivalently, that the polynomial $(X^n - 1)/(X - 1)$ is irreducible over $\mathbb{F}_2$. Finally the code must have a minimum distance large enough to ensure that *Decode* has a DFR less than $2^{-\lambda}$. This DFR is strongly conditioned by the statistical distribution of the number of errors remaining on the ciphertext after the use of the secret key. The coordinates of the error vector $\mathbf{e}' = \mathbf{x} \cdot \mathbf{r}_2 - \mathbf{y} \cdot \mathbf{r}_1 + \mathbf{e}$ are modeled as random independent Bernoulli variables of probability p^* , meaning that $\Pr[e'_k = 1] = p^*, \forall k \in \{0, \dots, n-1\}$. Note that p^* is only dependant on $n, w, w_{\mathbf{r}}$ and $w_{\mathbf{e}}$.

Parameters:
 $\mathbf{params} = (n, k, w, w_{\mathbf{r}}, w_{\mathbf{e}}, \mathbf{G} \in \mathbb{F}_2^{k \times n})$, where $\mathbf{G}$ is a generator matrix for the public code $\mathcal{C}$

KeyGen($\mathbf{params}$):
 $\mathbf{h} \xleftarrow{\$} \mathcal{R},$
 $sk = (\mathbf{x}, \mathbf{y}) \xleftarrow{\$} (\mathcal{S}_w^n(\mathbb{F}_2))^2,$
 $pk = (\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$

Encrypt($pk, \mathbf{m} \in \mathbb{F}_2^k$):
 $\mathbf{e} \xleftarrow{\$} \mathcal{S}_{w_{\mathbf{e}}}^n(\mathbb{F}_2),$
 $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} (\mathcal{S}_{w_{\mathbf{r}}}^n(\mathbb{F}_2))^2,$
 $ct = (\mathbf{u}, \mathbf{v}) = (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2, \mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e})$

Decrypt(sk, ct):
 $\mathbf{m}' = \mathcal{C}.Decode(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$

Fig. 1: The HQC cryptosystem.

2.2 Side-Channel Attacks

Side-channel attacks are a class of attacks introduced by Kocher et al. [24,25] against a processing unit that exploits an indirect observable of the system behavior to capture sensitive information when the system processes them. For instance, cache side-channel attacks take advantage of a difference in data access times between a cache hit (when the data is present in cache) and a cache

⁶ the attack [47] relative to the quasi-cyclic structure still applies.

miss (the data is only present in main memory), and have been widely used to break cryptographic software implementations [20,37,36,9,50,22,38,23].

Power-based side-channel attacks are a subclass that uses the power consumption to observe the system behavior. It has proven its effectiveness to significantly reduce the security level (including full key recoveries in the best case) of standard cryptographic algorithm implementations, including AES [42]. The side-channel attack is performed by collecting a given number (possibly large) of measurements called *traces*, and computing the correlation between a prediction *model* and the collected *traces*. The *trace* represents in practice a vector of data sampled from a sensor, and the *model* is a prediction of the power consumption due to the processing of an internal value in the cryptographic algorithm dependent on the input or the output, and a secret.

In practice, these attacks use the power variation of components that can be isolated with enough *traces* through statistical means, such as the registers of the CPU. When storing a new value into a register, the power consumption depends on the number of bit-flips produced, which can be expressed as the Hamming distance of the previous state (X) and the written state on the register (Y): $\omega(X \oplus Y)$. From this phenomenon, the leak produced by the writing on the register, noted $L(X, Y)$, can be deduced using the well known *leakage model* [8]:

$$L(X, Y) = \omega(X \oplus Y) + \mathcal{N}(0, \sigma),$$

where $\mathcal{N}(0, \sigma)$ is a Gaussian noise. As such, given enough *traces*, one can statistically determine if the measured power consumption is linearly linked to predicted values for some variables X and Y .

Although in some classical attacker models (for instance CCA), the adversary has legitimate access to the input or output of the cryptographic algorithm, an SCA attacker makes an assumption on the secret to perform predictions using the *model*.

For key recovery attacks, the internal value selected should be correlated to a sufficient number of key bits to reduce the number of guesses on the key. Depending on the algorithm and its implementation, the attack complexity can vary from a full key recovery with a single trace to a complexity close to brute force. In the following section, we present the best currently known attacks from side-channel information leakage on HQC.

3 Existing Side-Channel Attacks on HQC

This section presents existing side-channel attacks on the HQC version using Reed-Muller and Reed-Solomon codes (HQC-RMRS). The attacks mentioned here target the KEM version of HQC-RMRS (HQC-RMRS-KEM), either by retrieving the private keys $\mathbf{x}$ and $\mathbf{y}$ [15,46,20,43] or by retrieving the shared secret [14]. For the former, only key-recovery chosen-ciphertext side-channel attacks (KR-CCA-SCAs) on HQC-RMRS-KEM are publicly known, whilst the latter exploits the fact that the exchanged secret is used as a secret key to attack it.

3.1 Key Recovery CCA SCA

Power Consumption Analysis In the HQC version using BCH codes, KR-CCA-SCAs were already known, taking advantage of side-channel information leaking in particular during the decoding of the BCH code. Schamberger et al. introduced such an attack in [44], which was adapted in [43] to the HQC-RMRS. The method initially proposed and then adapted is the following: first, the ciphertext is set such that $\mathbf{u} = (1, 0, \dots, 0)$, which results in the decoder input $\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = \mathbf{v} - \mathbf{y}$.

Then $\mathbf{v}$ is chosen such that $\mathbf{v} - \mathbf{y}$ is at a distance of $d + 1$ from the all-zero codeword, where d is the minimal distance of the Reed-Muller Code. Then, an error is added to the entry of the decoder by controlling $\mathbf{v}$ to query each bit to learn if Reed-Muller decodes to the all-zeros codeword or not. This query, viewed as a response from an oracle, which is obtained by side-channel, allows to know if an error was removed or not, i.e. if a bit-flip occurred on a non-zero bit of $\mathbf{y}$ or not. Such an oracle can be obtained through multiple side-channels, e.g. power side-channel on hash functions or on the Reed-Solomon decoder.

This power analysis assumes the ability of the attacker to choose the ciphertext, which is not a requirement of our attacks. In comparison, our attacks are stealthier, as most of the times the process of adding errors will result in a wrong message decoding which can be spotted by the KEM when checking for successful decryption (in the context of [19]).

A more recent attack [6] leverages leaks happening during the Fast Hadamard Transform of the Reed-Muller decoding to recover the secret key. It is based on Soft Analytical Side Channel Attacks (SASCA) which combines side-channel information with Belief Propagation to find the full key. It also requires control over the ciphertext in order to set $\mathbf{u}$ to $(1, 0, \dots, 0)$.

Timing attacks on HQC Various time-based side-channel attacks on HQC-RMRS emerged, inspired by the work of Guo et al. [15], which led to a wide range of time-based attacks to apply the general strategy they introduced. The adversarial model of such strategies generally requires the attacker to be able to take control over the parameters $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{e}$, making him able to force the input of the Reed-Muller decoder to be either $\mathbf{mG} - \mathbf{y}$ or $\mathbf{mG} - \mathbf{x}$.

Using this idea, Guo et al. add an error $\mathbf{e}'$ such that $\mathbf{mG} - \mathbf{y} + \mathbf{e}'$ decodes to $\mathbf{c}'$, at a closer Hamming distance of the otherwise $\mathbf{c}$ codeword. Adding errors to $\mathbf{e}'$ bit by bit and being able to distinguish between a decoding execution that leads to $\mathbf{c}$ or $\mathbf{c}'$ then gives information on the error just like in the previous attack.

To distinguish between $\mathbf{c}$ and $\mathbf{c}'$, Guo et al. takes advantage of the HHK transform [19] of HQC that re-encodes the received message. During this encoding, the time taken to generate random polynomials depends on the seed, which is derived from the message. This step allows to compare different results of HQC-RMRS output just by measuring the time taken to re-encode the message, and thus, leaks if $\mathbf{c}$ equals $\mathbf{c}'$ or not.

The attacker model required for that technique is more restrictive than the one of [43], as it requires the additional assumption that the adversary is able to choose $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{e}$. Moreover, it suffers from the same issue: a large number of decoding failures will happen and may alert the victim.

This work pushed forward other timing-attacks on HQC-RMRS, such as [46], which takes advantage of modular reductions in the decoder of HQC that compile into the `div` assembly instruction. On some CPUs, such an instruction throughput varies depending on the input. The authors shown that it was possible to exploit the timing attack presented in [15] with a superscalar CPU, by adding a sibling thread to the Simultaneous MultiThreading (SMT) alongside the HQC execution thread. However, the specification of HQC-RMRS-KEM changed since then and now uses a salt as an input to the seed. Bad seeds could lead to hard-to-measure differences in `div` execution time. As such, the authors chose to brute-force the possible salt values in order to maximize the time taken by the `div` operation.

The adversarial model is even more restrictive for this technique than for the previous attacks as it requires, in addition to the modification of the parameters of the encoder, to be able to add a sibling SMT-thread to the HQC execution.

Polynomial sampling is another design flaw of HQC that can be exploited: when the victim generates $\mathbf{r}_1$ and $\mathbf{e}$, w_r bits are set to one, which involves w_r accesses to the cache. Knowing which cache lines were accessed allows to construct an oracle which returns whether two ciphertexts were decoded to the same message or not. This idea, brought in [20] gives an oracle that can be used to lead to the same attack as [15].

Improvements of KR-CCA-SCAs To improve the key-recovery techniques mentioned so far, further research should be focused on improving the method to choose the ciphertexts such that they would minimize the number of traces needed to succeed in the exploitation phase. This was tackled by Guo et al. in [15] that makes available a tool to choose such ciphertexts.

However, all of those attacks against HQC-RMRS-KEM still requires a model of the attacker with various strong hypothesis. At the very least, the attacker must be able to change $\mathbf{u}$ and $\mathbf{v}$ and generate potentially numerous invalid decryptions. In contrast, our attacks only requires an attacker to be able to replay legitimate ciphertexts to recover the private keys and works on the optimized (not only the reference) version of round 4 HQC-RMRS-PKE without SIMD instructions.

Comparison of KR-CCA-SCAs Table 2 sums up the theoretical number of oracle calls needed to fully perform the presented attacks as well as for our attack and specify the targeted HQC version submitted to the NIST. These oracle calls, though, does not take into account the reality of the experiments. Noise coming from previous or parallel computations as well as intrinsic noise coming from the probe may force the attacker to need multiple traces to perform the attack.

Our experimental proof-of-concept and the number of traces we need to fetch the secret key is thoroughly detailed in Section 6.

Work	Attack type	Oracle calls*	Target value	HQC implem.
[this work]	Power SCA	1	RM decoder input	Optimized, no SIMD
[6]	SASCA	1	RM decoder input	Reference
[46]	Division Instruction timing attack	1142	RM decoder output, Zero Tester Method	Optimized
[20,18]	Cache-timing + SCALDPC	9000	Polynomial sampling	Reference
[15,18]	Timing attack rejection sampling + SCALDPC	10000	Polynomial sampling	Optimized
[43]	Power SCA	52992	RM decoder output, Zero Tester Method	Reference
[20]	Cache-timing	53857	Polynomial sampling	Reference
[15]	Timing attack rejection sampling	866000	Polynomial sampling	Optimized

* Side-channel oracle with 100% accuracy

Table 2: Comparison of oracle calls needed for each attack against HQC-128

3.2 Shared-key recovery side-channel attack with profiling

While the goal of our attack is to recover the secret key $\mathbf{y}$, the main interest in HQC-RMRS nowadays is to be used in a key encapsulation mechanism. To attack the shared-key exchanged during the KEM phase, an electromagnetic side-channel attack can be used as in [14] to leak the shared key $\mathbf{m}$ by targeting the Galois field multiplication done in the Reed-Solomon decoder of the reference version of HQC. In this paper, the authors reduces the noise associated to the leakage model while diminishing the time needed in observation of the targeted device by profiling the leakage out of a copy of the target. [6] also contains this type of attack against the shared key, called a template profiling attack, as the attacker uses his own copy of the target to get as many traces as he wants to make a profiling of the distribution of the measured samples. This was unused in the other attacks presented, but may be helpful in the case of the attack presented here as it first needs a profiling of the target.

4 Key Recovery from Side-Channel Information

This section presents our attack on a mathematical level, and how the secret key elements $sk = (\mathbf{x}, \mathbf{y})$ can be retrieved from side-channel information. The

assumption is that side-channel information will act as an oracle that gives $\mathbf{mG} + \mathbf{e}' = \mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ to the attacker.

The HQC-RMRS decryption algorithm is structured as follows:

- `hqc_secret_key_from_string`: expands the secret key stored as a short string into $\mathbf{x}$ and $\mathbf{y}$.
- `vect_resize`: expands the $\mathbf{v}$ part of the ciphertext from code length $n_1 n_2$ to length n .
- `vect_mul`: computes the $\mathbf{u} \cdot \mathbf{y}$ product.
- `vect_add`: computes the sum $\mathbf{mG} + \mathbf{e}' = \mathbf{v} - \mathbf{u} \cdot \mathbf{y} = \mathbf{v} \oplus \mathbf{u} \cdot \mathbf{y}$.
- `reed_muller_decode`:
 - `expand_and_sum`: collects the codewords (the Reed-Muller code is duplicated 3 times for security category I, or 5 times for categories III and V) in blocks of bits.
 - `hadamard`: applies the Hadamard transform.
 - `find_peaks`: finalizes the decoding into $\mathbb{F}_2^8$ elements.
- `reed_solomon_decode`: locates and corrects the remaining errors, and output the decoded message of size λ .

Our attacks are based on leakage occurring during the `expand_and_sum` algorithm, which code is reminded in Figure 3. These attacks from power analysis allows to retrieve $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ bit-by-bit for each 16-bit block computed.

These attacks unfold as follow:

1. Calibration phase: The attacker finds the Points of Interests (POI) in the power consumption traces. This can be done with passive observation of the target during legitimate messages decryption from any source, or by observing the execution of the decryption on a computer with architecture similar to the target (profiling).
2. Active phase: The attacker observes the decryption of a legitimate ciphertext and replays it a specific number of times to collect power consumption traces.
3. Analysis phase: From the traces, the attacker determines the statistically most probable $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$. The impact of the number of traces on the success of the attack is detailed in Section 6.4.

Given $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ from this method modeled by an oracle, the attacker will retrieve the $\mathbf{y}$ of the secret key. This is the only part of the key required for decryption of subsequent legitimate ciphertexts. Knowing the replayed ciphertext $ct = (\mathbf{u}, \mathbf{v})$ the attacker first computes $\mathbf{v} - (\mathbf{v} - \mathbf{u} \cdot \mathbf{y}) = \mathbf{u} \cdot \mathbf{y}$. Then, in the case where $\mathbf{u}$ is invertible in $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, he computes $\mathbf{u}^{-1} = \gcd(\mathbf{u}, X^n - 1)$, and then retrieves the secret key by computing $\mathbf{u}^{-1} \cdot (\mathbf{v} - (\mathbf{v} - \mathbf{u} \cdot \mathbf{y})) = \mathbf{y}$. This method can be extended as follows in order for the attacker to still retrieve $\mathbf{y}$ while $\mathbf{u}$ is not invertible.

Using [40, Th 3.6.5] on matrix $\mathbf{rot}(\mathbf{u})$, it is possible to decompose $\mathbf{rot}(\mathbf{u})$ as $\mathbf{P} \times [\mathbf{I}_r : \mathbf{C}] \times \mathbf{Q}$ where r is the rank of $\mathbf{rot}(\mathbf{u})$, $\mathbf{P}$ is a matrix of dimension $n \times r$, $\mathbf{Q}$ is a permutation matrix and $\mathbf{C}$ is a matrix of order $r \times (n - r)$. Using the fact that $\mathbf{P}$ has linearly independent columns and noting $\mathbf{P}_L^{-1}$ his left inverse, it is possible to continue with the key recovery by computing:

$$\begin{aligned}
(\mathbf{v} - (\mathbf{v} - \mathbf{u} \cdot \mathbf{y})) \times (\mathbf{P}_L^{-1})^\top &= (\mathbf{v} - (\mathbf{v} - \mathbf{y} \times \mathbf{rot}(\mathbf{u})^\top)) \times (\mathbf{P}_L^{-1})^\top \\
&= (\mathbf{y} \times \mathbf{rot}(\mathbf{u})^\top) \times (\mathbf{P}_L^{-1})^\top \\
&= (\mathbf{y} \times \mathbf{Q}^\top \times [\mathbf{I}_r : \mathbf{C}]^\top \times \mathbf{P}^\top) \times (\mathbf{P}_L^{-1})^\top \\
&= (\mathbf{y} \times \mathbf{Q}^\top \times [\mathbf{I}_r : \mathbf{C}]^\top) \times (\mathbf{P}_L^{-1} \times \mathbf{P})^\top \\
&= \mathbf{y}' \times [\mathbf{I}_r : \mathbf{C}]^\top
\end{aligned}$$

where $\mathbf{y}' = \mathbf{y} \times \mathbf{Q}^\top$. Doing so reveals to the attacker r bits of $\mathbf{y}$ (which are the same as $\mathbf{y}'$, not in the same order). While the $n - r$ last bits should be brute-forced, the order of the bits can be retrieved by left-multiplying by $\mathbf{Q}^{-1} = \mathbf{Q}^\top$.

Being able to retrieve r bits of the secret value $\mathbf{y}$, it is however important to know if brute-forcing the $n - r$ last bits is possible or not.

In the following $\mathbf{rk}(\cdot)$ denotes the rank of a matrix. The end of this section is dedicated to understanding the scope of these attacks. To do so, we need the following lemma, which is proved in Appendix B:

Lemma 1. *Let $\mathbf{u}$ be an element of $\mathcal{R}$ and let $\mathbb{1} = 1 + X + \dots + X^{n-1}$. Then:*

$$\mathbf{rk}(\mathbf{rot}(\mathbf{u})) = \begin{cases} 0 & \text{if } \mathbf{u} = 0 \\ 1 & \text{if } \mathbf{u} = \mathbb{1} \\ n-1 & \text{if } \mathbf{u} \neq 0 \text{ and } \omega(\mathbf{u}) \text{ is even} \\ n & \text{otherwise, i.e., if } \mathbf{u} \neq \mathbb{1} \text{ and } \omega(\mathbf{u}) \text{ is odd} \end{cases}.$$

Using the above Lemma 1, an attacker is highly unlikely to come across the first two cases. A simulation (available in the supplementary materials [27]) of the HQC generation of $\mathbf{u}$ and computation of the rank of $\mathbf{rot}(\mathbf{u})$ on 4,600,000 entries confirms that none of them led to generation of $\mathbf{u} = 0$ or $\mathbf{u} = 1 + X + \dots + X^{n-1}$. However, half of the $\mathbf{u}$ were invertible (2,300,148 out of 4,600,000) and the other half leaked $17,668 = n - 1$ bits. For those attacks, we conclude that, at worst, only 1 bit has to be brute-forced.

5 Target description and associated power model

The power analysis model is extracted from an abstraction layer stemming from the behavior of the targeted system, itself derived from the description of the target. While an high level description model of the target can lead to portable side-channel attacks (i.e. independent from the target hardware), a model too distant from the hardware may miss exploitable contributions to the power consumption. To make this attack portable while keeping sufficient effectiveness, we targeted the processor ISA as the abstraction layer of the system. This choice is motivated by:

1. the wide usage of processor-based architectures in practice, from System on Chips (SoC) to general purpose computers;
2. the fact that the ISA is in general well documented;
3. the possibility to cover all families of CPUs sharing the same ISA.

Among the most popular ISAs are the ones from Arm (AArch64, AArch32, ARM T32), the x86-64 from Intel, as well as the open standard RISC-V. In the following, we provide full key recovery attack targeting the RISC-V ISA and an attack on an Arm Cortex-M4 SoC, which implements the ARMv7-M, which is built upon the ARM T32 ISA, along with a complete analysis of attack portability to other ISAs.

5.1 Description of the attack

Source of the side-channel leakage The proposed attack targets the `expand_and_sum` operation in the optimized implementation of Reed-Muller decoding, and in particular the expansion part. Figure 3 provides the C code of the optimized no SIMD version of the Reed-Muller `expand_and_sum` function. During the expansion phase, all the bits of the error $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ are extracted and stored separately into a vector (see line 16 in Figure 3). Since micro-processors only operates on bytes, four successive operations are required:

1. A load operation of (some) bytes of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ from memory to a register of the processor.
2. A logical right shift of the loaded bytes to position the bit to be extracted into the less significant bit.
3. A logical AND with 1 to set to zero all the bits except the less significant one.
4. A store operation to write the extracted bit back to the memory.

Note 1: All tested architectures (i.e. ARM T32, RISC-V, and x86-64) and standard gcc compiler optimization levels (00, 01, 02, 03, 0g, 0z) without vector extensions of the ISA, the enumerated operations have been translated into 1 instruction per operation in the same order (i.e. no multiple operations have been factored into a single instruction). The only specific case is for x86-64 with 03 optimization level and SIMD capable target, which use vectored operations instead and a full loop inlining.

Note 2: For the gcc optimization levels 00 and 0g for x86-64, the compiler reproduces the register's usage of RISC architectures. This will be the main assumption in the following. With other optimization levels for x86-64, the same register is used for operation 1 and 2. This case will be discussed in Section 5.3.

These observations make it possible to model the expand operation into 4 generic instructions without reducing the attack portability. For convenience, we will adopt the RISC-V assembler syntax. Lets consider a_0 and a_1 two registers storing respectively the memory address of (some) bytes of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ and the memory address of the destination to store the bit extracted; t_0 and t_1 two registers used to store intermediate results, and r a register storing the number of bit required for the shift operation. The 4 operations can be translated into

the following instructions to model the inner loop of `expand_and_sum` (i.e. line 16 in Figure 3):

1. `lw t0, 0 (a0)` $\leftarrow$ Load operation from memory
2. `srl t1, t0, r` $\leftarrow$ right shift of the loaded bytes
3. `andi t1, t1, 1` $\leftarrow$ bit extraction
4. `sh t1, 0 (a1)` $\leftarrow$ Store the extracted bit into memory

Fig. 2: Instruction flow of the inner loop of the first loop in the `expand_and_sum`

```

1 // codeword is 128 bits, seen multiple ways
2 typedef union {
3     uint16_t u16[8];
4     uint32_t u32[4];
5 } codeword;
6
7 // Expanded codeword is 16*128 bits, seen multiple ways
8 typedef union {
9     int16_t i16[128];
10 } expandedCodeword;
11
12 void expand_and_sum(expandedCodeword *dst, codeword src[]) {
13     // start converting the first copy
14     for (size_t part = 0; part < 8; part++) {
15         for (size_t i = 0; i < 16; ++i) {
16             dst->i16[(part << 4) + i] = src->u16[part] >> i & 1;
17         }
18     }
19     // sum the rest of the copies
20     for (size_t copy = 1; copy < MULTIPLICITY; copy++) {
21         for (size_t part = 0; part < 8; part++) {
22             for (size_t i = 0; i < 16; ++i) {
23                 dst->i16[(part << 4) + i] += src[copy].u16[part] >> i &
24                 1;
25             }
26         }
27     }
28 }

```

Fig. 3: C code of the Reed-Muller `expand_and_sum` function's optimized version with no SIMD.

Notice that instruction 1 is not explicitly in the inner loop because a_0 is not modified between iteration steps (i.e. the operation is performed 16 times on the same portion of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$). a_1 is incremented (by 2 for instance) to store the bit extracted into separate memory bytes, and r is incremented by 1 which allows an attacker to extract bit-by-bit the bits of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$.

As the `expand_and_sum` processes $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ by blocks of 16 bits, we only present the side-channel extraction of one particular block represented by the bit sequence $b = b_{15} b_{14} \dots b_0$.

To take into account the dynamic evolution of the registers' content over time, we model registers t_0 , t_1 and r as functions of the time t . Since the processor's registers are synchronous to the processor's clock of period T , the bit flipping only occurs at clock ticks, i.e. αT for some $\alpha \in \mathbb{N}$. For instance, given $\alpha \in \mathbb{N}$ and a measurement period of $\epsilon \in \mathbb{R}$ ($\epsilon \ll T$) that covers the transitional phase for the bit-flip to occur at the tick αT , the transition leakage of a register t_i at clock cycle αT can be calculated as:

$$\text{HD}(t_i(\alpha T - \epsilon), t_i(\alpha T + \epsilon)) \quad (1)$$

For a particular iteration $it \in \{0, \dots, 15\}$ of the `expand_and_sum` function and instruction i , we note $c_{it,i}$ the clock cycle when the result of instruction i is committed to the destination register.

The first possible attack strategy is to go through the bit-stream in reverse order. By first extracting the most significant bit using the AND operation leakage. Then it is possible attack the rest of the bits by taking advantage of the leakage stemming from the SRL execution: the register t_1 changes state from the result of the previous AND to the right-shifted state of the current iteration. The following details how to recover the whole 16-bits block bit-by-bit.

Extraction of the Most Significant Bit b_{15} The most significant bit can be straightforwardly extracted in the transition leakage model by analysing the second-to-last iteration of the loop, i.e. when performing the extraction of b_{14} . After processing the shift operation, register t_1 contains the value of the bit sequence b right-shifted by 14, i.e. $0 \dots 0 b_{15} b_{14}$. Then, after the AND operation, $0 \dots 0 b_{14}$ is stored in the same register t_1 . When applying the transition leakage model to t_1 , the commitment of the AND operation to t_1 , results in the attacker observing the Hamming distance between t_1 before the commitment and after the commitment, i.e.:

$$\begin{aligned} \text{HD}(t_1(c_{14,3} - \epsilon), t_1(c_{14,3} + \epsilon)) &= \text{HD}(0 \dots 0 b_{15} b_{14}, 0 \dots 0 b_{14}) \\ &= \text{HW}(0 \dots 0 b_{15} b_{14} \oplus 0 \dots 0 b_{14}) \\ &= \text{HW}(0 \dots 0 b_{15} 0) \\ &= b_{15} \end{aligned} \quad (2)$$

Since the power model only depends on the value of b_{15} , a distinguisher based on a threshold of the power consumption's amplitude when the commitment occurs is sufficient to extract the bit value.

Notice that the AND operation during the last loop iteration do not provides information on any bit of b . After the right-shifting by 15 bits, the bit sequence of t_1 is $0 \dots 0 b_{15}$, invariant during the AND operation.

Extraction of Bit b_{14} to b_0 The extraction of the other bits of b relies on the persistence of the registers' value between loop iterations. For an iteration

it ($it > 0$), register t_1 only contains the bit extracted in the previous iteration (i.e. $0 \cdots 0 b_{it-1}$). For instance, b_{14} can be extracted when the right-shifting operation occurs during the last iteration (i.e. $it = 15$):

$$\begin{aligned} \text{HD}(t_1(c_{15,2} - \epsilon), t_1(c_{15,2} + \epsilon)) &= \text{HD}(0 \cdots 0 b_{14}, 0 \cdots 0 b_{15}) \\ &= \text{HW}(0 \cdots 0 b_{14} \oplus 0 \cdots 0 b_{15}) \\ &= b_{14} \oplus b_{15} \end{aligned} \quad (3)$$

Since b_{15} can be deduced independently of the other bits, b_{14} can be distinguished using a conditional threshold that only depends on b_{15} value.

To extract the other bits of b , it is sufficient to target the same instruction (i.e. right-shifting) for each iteration in the reverse order. Knowing b_{15} and taking the case of the 14th bit as the base case, the same can be applied for bit b_{13} as the transition leakage of the right-shift operation at iteration 15 is equal to:

$$\begin{aligned} \text{HD}(t_1(c_{13,2} - \epsilon), t_1(c_{13,2} + \epsilon)) &= \text{HD}(0 \cdots 0 b_{13}, 0 \cdots 0 b_{15} b_{14}) \\ &= b_{15} + b_{13} \oplus b_{14} \end{aligned} \quad (4)$$

Since b_{14} and b_{15} are known, then b_{13} can be deduced using equation 4. An attacker can then proceed with an induction step using the property below stemming from the reasoning that led to leak b_{13} applied to any iteration $it < 14$:

$$\begin{aligned} \text{HD}(t_1(c_{it,2} - \epsilon), t_1(c_{it,2} + \epsilon)) &= \text{HD}(0 \cdots 0 b_{it}, 0 \cdots 0 b_{15} \cdots b_{it+1}) \\ &= \sum_{i=it+2}^{15} b_i + b_{it} \oplus b_{it+1} \end{aligned} \quad (5)$$

This way, one can match his measurement against the Hamming distance presented for each bit which allows him to recover the whole block of 16-bits, and by repetition, the value of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$.

5.2 Simulation method

To generate execution traces at the ISA level based on the Hamming distance power model, we developed a simulator. The simulator takes as input a binary written for a RISC-V CPU and executes the instructions step-by-step, simulating a RISC-V CPU with, in particular, a simulation of the registers, the RAM, the ROM and a level of cache. The ISA simulated is the RISC-V one due to its increasing adoption. Moreover, as RISC-V is part of the RISC (Reduced Instruction Set Computer) architecture family and an open-source standard, it is easier to make a simulator compliant to this ISA compared to the CISC (Complex Instruction Set Computer) ones. This simulator is openly available at the gitlab instance [31]. It is based on the RISC-V workbench presented in [35] which is also hosted online [32]. While at first the simulator was used for McEliece cache processing optimizations, for this work, the simulator was reworked to add the

leakage model to the simulated CPU. The forked simulator makes it possible to easily check for side-channel leakage for each instruction in the Hamming distance leakage model by running a statistical t-test with usual parameters [13,45] or by extracting the CPU state at each step of the execution to run custom attacks. That simulator was thus tailored to add a proof-of-concept of the presented leakage. To reduce simulation time, we only executed the Reed-Solomon decode that contains the exploitable `expand_and_sum` function presented in Figure 3.

As the leakage modeled in the simulator stems only from the architecture, the adopted leakage model only depends on the Hamming distance between the previous content of a register and what overwrites it, without noise, i.e. the leakage model described in Section 2.2 simplifies to: $L(X, Y) = \omega(X \oplus Y)$.

The method just detailed can be exploited with the simulator against the RISC-V architecture. To do so, the set of leaking instructions for each bit pertaining to the `AND` and `SRL` was easily retrieved thanks to the simulator step-by-step execution design which allowed us to unfold the experiment.

5.3 ISA attack portability

To evaluate the attack’s portability, we evaluated the assembly code produced from the optimized `expand_and_sum` function for 3 different ISAs: two from the RISC family, i.e. ARM T32 and RISC-V; and one from the CISC family, i.e. x86-64. They represent a wide range of target systems, covering from connected objects to general purpose computers, and which are known to be major platforms [33]. Since the assembly code may slightly vary between compiler versions, we provide the complete configuration to reproduce the experimental results. Figure 4 provides the assembly code produced for the RISC ISAs, which follow the same instruction flow as shown in Figure 2. Hence, as the compilation strategy remains the same, it fits perfectly with the attack model defined previously.

The previous compilation strategy also holds for x86-64 with optimization level `0g` or `00`. However, with optimization level `01` or `02`, there is a slight difference for the right-shift operation: the source and destination registers are the same for the right-shifting (`shr`) instruction, as highlighted in red in Figure 5.

Therefore, we have to adapt the strategy proposed to extract the bits from b_{14} to b_0 since both registers t_0 and t_1 are the same in that case. Nevertheless, it is still possible to take advantage of the leakage stemming from the `AND` instruction in the same way that it was used to recover the 15th bit. This variant of the attacker model can be used on RISC architectures but is also portable to the specific cases brought by the x86-64 ISA. This also shows that different measurement of the power consumption may leak information, which gives more opportunity to an attacker. The revisited transition leakage for this specific architecture for $it > 0$ is presented below:

$$\begin{aligned} \text{HD}(t_1(c_{it,2} - \epsilon), t_1(c_{it,2} + \epsilon)) &= \text{HD}(b_{15} \dots b_1 b_0, 0 \dots 0 0 b_{it}) \\ &= \sum_{i=1}^{15} (b_i) + b_{it} \oplus b_0 \end{aligned} \quad (6)$$

ARM T32

gcc version: 10.3.1 binutils version: 2.36 compiler flags: -O2

```
ldrh.w  r7,    [r6,    #2]!      (1)
add.w   r5,    r0,    r2,    lsl #1
movs    r3,    #0
asr.w   r4,    r7,    r3          (2)
adds    r3,    #1
and.w   r4,    r4,    r4,    #1   (3)
cmp     r3,    #16
strh.w  r4,    [r5],    #2       (4)
```

RISC-V

gcc version: 14.2.0 binutils version: 2.43.1 compiler flags: -O2
-march=rv32g -fno-builtin

```
lw      a2,    0(t4)              (1)
mv      a3,    t3,
li      a5,    0
srl     a4,    a2,    a5          (2)
andi    a4,    a4,    1          (3)
sh      a4,    0(a3)            (4)
```

Fig. 4: Comparison of the assembly code produced for the first `expand_and_sum` function loop inner loop, for ARM T32 and RISC-V ISAs. The numbered instructions in bold refer to the high level instructions required for the ISA attack.

Domain of applicability of the attack As shown, it is possible to reuse the presented attack strategy, with either of the variant of the attack. To evaluate the domain of applicability of the attack, standard architectures have been analyzed, especially RISC and CISC architectures, with a distinction between Load-Store and non-Load-Store ones, for several compiler optimization levels. For instance, in Load-Store architecture, dedicated instructions are required to perform memory data loads and stores, which can impact to code generated.

Figure 4 makes clear that the Load-Store paradigm is also used on x86-64 for optimized compilation in our case. As such, the presented attack is not only applicable on every SoC implementing the RISC-V ISA, but also on all the SoCs implementing the most commons ISAs, i.e. x86-64, RISC-V and the ARM T32. Moreover, as even the aggressive optimizations of -O3 still compiles to the shown assembly for the RISC-V and ARM cases, this attack can be applied to most binaries of HQC.

As a non-exhaustive comparisons of main platforms, Table 3 shows whether a given ISA is impacted by this attack based on the first method in Section 5.1 (named V0 in the table) or if it is vulnerable to the variant proposed for the portability of the attack in Section 5.3 (named V1 in the table). The dash "-" in the RISC-V case denotes a case that wasn't tested as the `expand_and_sum` was not ported for the specific `Oz` compilation strategy. In the `O3` x86-64 case the option `-fno-tree-vectorize` was added to avoid auto-vectorization from the

x86-64

gcc version: 14.2.1 binutils version: 2.44 compiler flags: -O2

```
movzbl  (%r9,    %r10,  1),  %r8d    (1)
xor     %ecx,   %ecx
nopw    0x0     (%rax,   %rax, 1)
mov     %r8d,   %eax
shr     %cl,    %eax                (2)
and     $0x1,   %eax                (3)
mov     %ax,    (%rdi,   %rcx, 2)    (4)
```

Fig. 5: Assembly code produced for the first `expand_and_sum` function loop inner loop for the x86-64 ISAs. The right shift (in red) is done on the same register, as opposed to other ISA assembly code.

compiler. In the ARM case, the tests were made against AArch32 instead of the ARM T32 as the official ARM gcc toolchain is discontinued for the latter [5].

ISA	Optimization flag					
	-Og	-O0	-O1	-O2	-O3	-Oz
RISC-V	V0/V1	V0/V1	V0/V1	V0/V1	V0/V1	-
AArch32	V0/V1	V0/V1	V0/V1	V0/V1	V0/V1	V0/V1
x86-64	V0/V1	V0/V1	V1	V1	V1	V1

Table 3: Tested ISAs against our attack variants

6 Power analysis attack

6.1 Target description and experimental setup

In this section, the leakage has been evaluated in a real hardware: a STM32F303 target embedding a ARM Cortex-M4. We show that a real target may add leakage contribution coming from the micro-architectural implementation of the said ISA.

All the experiments have been carried out on the NewAE ChipWhisperer Pro, a well-known test-bench for power-based side-channel attacks and glitch fault injection allowing fast and reproducible experiments. The ChipWhisperer module act as an interface between a host computer (connected with USB) and the target (i.e. the STM32F303); and is capable to program, communicate and capture the power consumption of the target. The capture itself is based on Python scripts while the analysis of them is based on SageMath scripts (both of them are available online at [27]). We used default ChipWhisperer configuration

except for the Analog to Digital Converter gain, which has been tuned to take advantage of the full range.

Comparing to the previous leakage model used by the simulator that took into account only the architectural level of the SoC (i.e. the ISA), the leakage model evolves here to match what was used in [8]: the micro-architectural effects are being considered as a Gaussian noise added to the power leakage due to the bit-flips. Our experiments show that this leakage model is enough to fully recover the secret key $\mathbf{y}$ from HQC with only a few traces.

6.2 Experiments and Results

Locating the Points of Interest The first experiment was designed to collect traces of the execution of the `expand_and_sum` loop for each value of the 16-bit block, meaning a total of 65536 traces, over 1000 samples, which took a total runtime of approximately 34 minutes and 526 Megabytes.

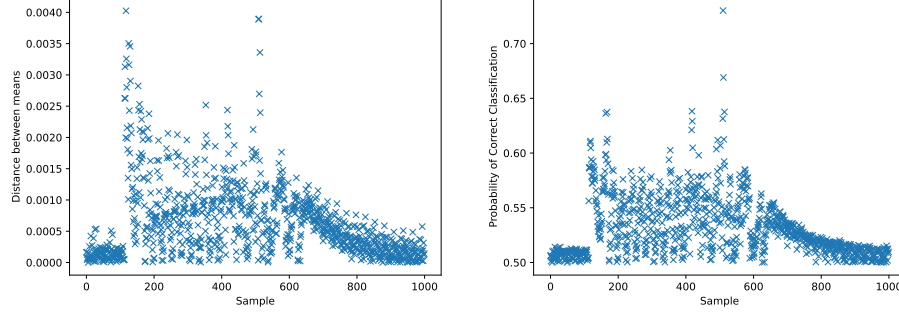
For each bit, traces are sorted according to the value of the bit (32768 traces each). For each sample the mean and standard deviation of the distribution of power consumption values is computed $(\mu_0, \sigma_0, \mu_1, \sigma_1)$.

The attacker will then select specific samples such that the distributions $\mathcal{N}(\mu_0, \sigma_0)$, and $\mathcal{N}(\mu_1, \sigma_1)$ are the most separated possible. This makes these samples good predictors of whether the measured power consumption is the result of the computation of a 0 or a 1. Among all the available samples, one is chosen as a POI for being the best predictor, or chosen arbitrarily in case of multiple samples being equivalent predictors.

Although the difference $|\mu_0 - \mu_1|$ is a good value to consider in order to detect the POI, as it can be seen in Figure 6a, a more precise differentiation (presented in Section 6.3) based on the noise being model as normal can be made to increase correct classification chances (Figure 6b) and select the best POI. It is particularly well seen on bit_{10} where despite multiple samples leaking leak information (shown by the distance between the means) variations in the noise levels do not make them the best POI in terms of probabilities of correct classification. A standard Signal-to-Noise ratio over time is given in Appendix C.

Confirming the noise model The distributions at the identified POI fit tightly the expected leakage model $L(X, Y) = \omega(X \oplus Y) + \mathcal{N}(0, \sigma)$. For illustration, Figure 7 shows the histograms of collected power values at the bit_0 POI, compared to the estimated normal distributions.

Given the noisy nature of the measurements, with a fixed number of traces to determine POIs from, it is possible that the selected ones are not the optimal ones. Figure 8 shows the evolution of the POIs of bit_0 and bit_{11} according to the number of traces collected. These bits were chosen as they represent the fastest and slowest convergence to the optimal POI. From Figure 8b we observe that 2^{10} traces are sufficient to converge for this noise level. This means that the template profiling of the calibration phase can be done in 32 seconds. It can be done in less time but at the cost of more traces needed during the active phase to compensate for the worse predictors.



(a) Distance between the means of the traces when bit_{10} is set to 0 or 1 (b) Probability of correct classification of bit_{10} for each sample

Fig. 6: Differences of metrics in the election of the POI

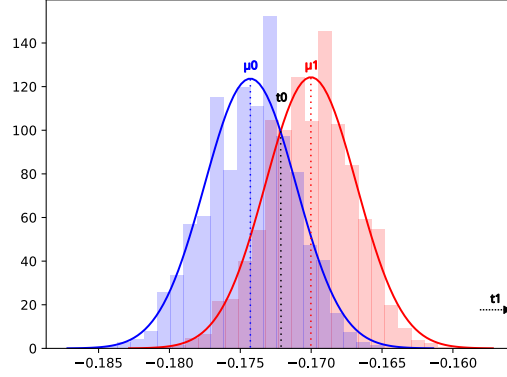


Fig. 7: Samples distributions for 0 and 1 on the POI of bit_0

6.3 Statistical model of the results

In HQC, the bits of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ are supposed to all be independent and have an equal chance to be a 0 or a 1. Given a single power consumption measurement x at the POI of a given bit, the attacker best option is to find a threshold t , such that whether x is above or below t gives the best guess for the bit. This threshold can be found given the hypothesis that samples are taken from normal distributions such that the probability for a sample x is:

$$\frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left(-\frac{(x - \mu_0)^2}{2\sigma_0^2}\right) \text{ or } \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(x - \mu_1)^2}{2\sigma_1^2}\right)$$

Given the prior knowledge of the characteristics $\mu_0, \sigma_0, \mu_1, \sigma_1$, the attacker finds the threshold t since it is the value for which the distributions are equal:

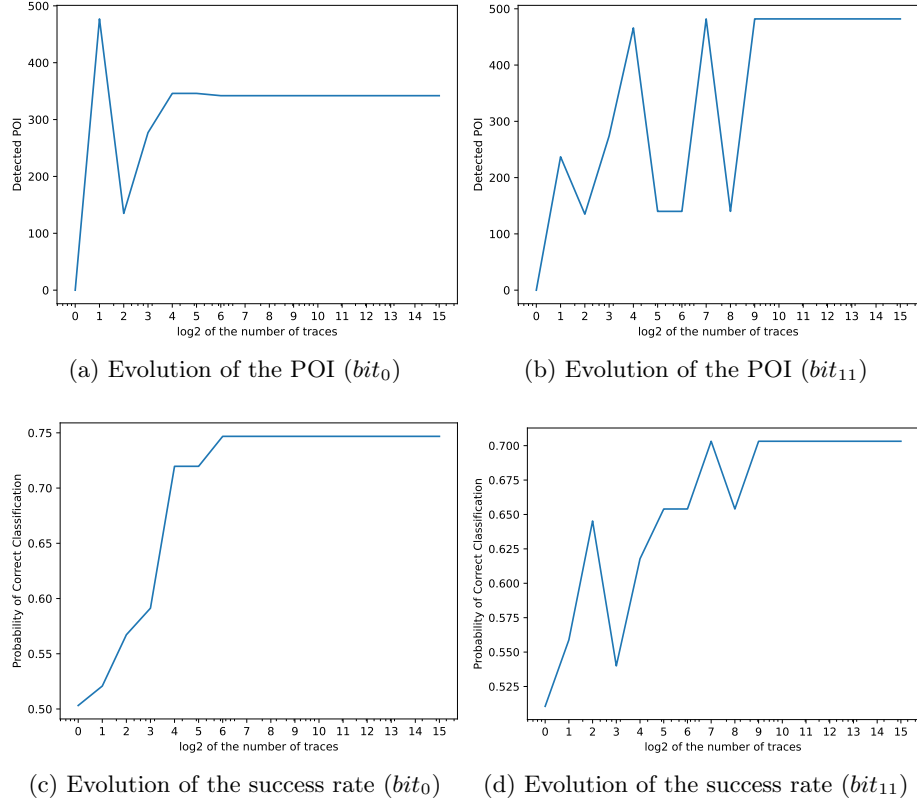


Fig. 8: Evolution of the POI and the probability of correct classification with the number of traces

$$\frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left(-\frac{(t-\mu_0)^2}{2\sigma_0^2}\right) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(t-\mu_1)^2}{2\sigma_1^2}\right)$$

For a better accuracy of its prediction, the attacker can take S samples $(x_1, \dots, x_S)$ and average them. The distribution of the means have μ_0 and μ_1 , but variances σ_0^2/S and σ_1^2/S , which changes the threshold to the solution:

$$\frac{1}{\sqrt{2\pi\sigma_0^2/S}} \exp\left(-\frac{(t-\mu_0)^2}{2\sigma_0^2/S}\right) = \frac{1}{\sqrt{2\pi\sigma_1^2/S}} \exp\left(-\frac{(t-\mu_1)^2}{2\sigma_1^2/S}\right)$$

Then, in order to solve for t , the logarithm application gives:

$$\ln\left(\frac{1}{\sqrt{2\pi\sigma_0^2/S}}\right) - \frac{(t-\mu_0)^2}{2\sigma_0^2/S} = \ln\left(\frac{1}{\sqrt{2\pi\sigma_1^2/S}}\right) - \frac{(t-\mu_1)^2}{2\sigma_1^2/S}$$

Which can be rewritten to:

$$-\frac{1}{2} \ln(2\pi\sigma_0^2/S) - \frac{(t-\mu_0)^2}{2\sigma_0^2/S} = -\frac{1}{2} \ln(2\pi\sigma_1^2/S) - \frac{(t-\mu_1)^2}{2\sigma_1^2/S}$$

From this we deduce that the following should hold:

$$\begin{aligned} \ln \frac{\sigma_0^2}{\sigma_1^2} &= 2 \left(\frac{(t-\mu_1)^2}{2\sigma_1^2/S} - \frac{(t-\mu_0)^2}{2\sigma_0^2/S} \right) \\ &= \frac{(t-\mu_1)^2}{\sigma_1^2/S} - \frac{(t-\mu_0)^2}{\sigma_0^2/S} \\ &= S \left(\frac{(t-\mu_1)^2}{\sigma_1^2} - \frac{(t-\mu_0)^2}{\sigma_0^2} \right) \\ &= S \left(\frac{t^2 - 2\mu_1 t + \mu_1^2}{\sigma_1^2} - \frac{t^2 - 2\mu_0 t + \mu_0^2}{\sigma_0^2} \right) \\ &= S \left(\left(\frac{1}{\sigma_1^2} - \frac{1}{\sigma_0^2} \right) t^2 - \left(\frac{2\mu_1}{\sigma_1^2} - \frac{2\mu_0}{\sigma_0^2} \right) t + \left(\frac{\mu_1^2}{\sigma_1^2} - \frac{\mu_0^2}{\sigma_0^2} \right) \right) \end{aligned}$$

Thus, the desired threshold is:

$$t = \frac{-S \left(\frac{2\mu_0}{\sigma_0^2} - \frac{2\mu_1}{\sigma_1^2} \right) \pm \sqrt{\Delta}}{2S \left(\frac{1}{\sigma_1^2} - \frac{1}{\sigma_0^2} \right)},$$

where

$$\Delta = S^2 \left(\frac{2\mu_0}{\sigma_0^2} - \frac{2\mu_1}{\sigma_1^2} \right)^2 - 4S \left(\frac{1}{\sigma_1^2} - \frac{1}{\sigma_0^2} \right) \left(S \left(\frac{\mu_1^2}{\sigma_1^2} - \frac{\mu_0^2}{\sigma_0^2} \right) - \ln \frac{\sigma_0^2}{\sigma_1^2} \right).$$

The discriminant can be considered to be strictly positive in our case. As the parameters of the distributions are determined from precise experimental values, it is highly improbable that they share exactly the same standard deviation, which would cause the curves to have either a single point of intersection, or infinitely many if they also have the same mean.

With t_0, t_1 the roots, we can suppose without loss of generality that $\mu_0 < t_0 < \mu_1 < t_1$, as illustrated for *bit*₀ in Figure 7. With Φ being the cumulative distribution function of the normal distribution, the probabilities P_{ij} that the attacker guesses the value i from a bit of value j are:

$$\begin{aligned} P_{00} &= \Phi \left(\frac{t_0 - \mu_0}{\sigma_0/\sqrt{S}} \right) + \left(1 - \Phi \left(\frac{t_1 - \mu_0}{\sigma_0/\sqrt{S}} \right) \right) \\ P_{01} &= \Phi \left(\frac{t_0 - \mu_1}{\sigma_1/\sqrt{S}} \right) + \left(1 - \Phi \left(\frac{t_1 - \mu_1}{\sigma_1/\sqrt{S}} \right) \right) \\ P_{10} &= \Phi \left(\frac{t_1 - \mu_0}{\sigma_0/\sqrt{S}} \right) - \Phi \left(\frac{t_0 - \mu_0}{\sigma_0/\sqrt{S}} \right) \end{aligned}$$

$$P_{11} = \Phi\left(\frac{t_1 - \mu_1}{\sigma_1/\sqrt{S}}\right) - \Phi\left(\frac{t_0 - \mu_1}{\sigma_1/\sqrt{S}}\right)$$

The total chances of correctly guessing a bit P_{bit} given that ones and zeroes have the same chance of appearing are:

$$P_{bit} = \frac{1}{2} (P_{00} + P_{11}) \quad (7)$$

Given that the operations are done in successive 16 bits blocks, the probabilities that all bits are guessed correctly is

$$P_{block} = \prod_{i=0}^{15} P_{bit_i}$$

Finally, the probability that all $\lceil \frac{n}{16} \rceil$ blocks are guessed correctly is

$$P_{all} = P_{block}^{\lceil \frac{n}{16} \rceil}$$

The success probability of the attack for our measurements is presented in Figures 9a and 9b.

6.4 Statistical analysis of the results

To illustrate with the experimental values, the probability of guessing correctly a bit from a single trace is $0.6996 \leq P_{bit_i} \leq 0.7616$ depending on the bit, the probability of guessing a 16-bit block is $P_{block} \simeq 0.006$, and the probability of guessing the full vector in category I is $P_{all} \simeq 9.558 \times 10^{-2464}$. With 48 traces, $0.9998 \leq P_{bit_i}$ for all bits, $P_{block} \simeq 0.9994$ and $P_{all} \simeq 0.50$.

It can be noticed from Figure 9a that in order to have a success rate over 50%, 48 to 56 traces are required depending on the security category, and with 83 traces the success rate is always over 99%. Note that this is equivalent to 2.5 seconds with our experimental setup.

Figure 9b shows more clearly for very high or low number of traces, and show that the attacker should be discouraged to try the attack with too few traces, but quickly has high probabilities of success with a few more.

The observed curve has a slope that is strongly linked to the noise levels, while n only creates a vertical offset. It can be seen in Figure 10 that halving the noise levels of the measurements, for instance with a more precise measuring tool or more favorable experimental setting, makes it such that 20 traces already gives a high probability of success of the attack. However, by increasing the noise by half, the required number of traces for 50% success rate is 125. This gives an insight on how the complexity of the attack may change on more simple or more complex architectures.

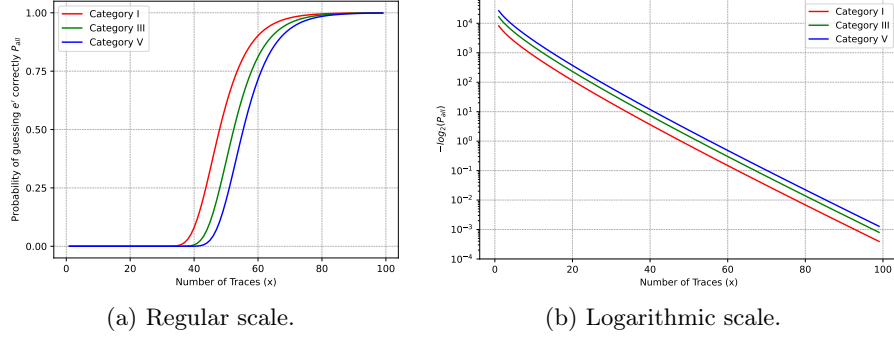


Fig. 9: Probabilities that $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ is guessed correctly according to the number of traces and security category.

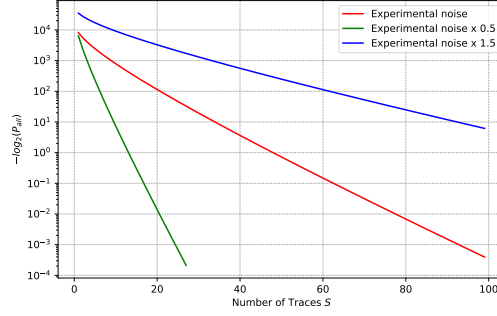


Fig. 10: Logarithm of the probabilities that $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ is guessed correctly according to the number of traces and various noise levels for security category I (log scale).

7 Countermeasures evaluation

The first solution to limit the applicability of these attacks is to change the way `expand_and_sum` manipulates the data. One could avoid the problematic reuse of registers with explicit assembly instructions preventing the ISA-level attack. However this technique would require different instructions for each ISA, which is not portable. For the same reason, creating auxiliary variables or inserting useless operations between the load, the shift and the AND cannot be proven to be efficient for all ISAs and all compilers and optimization levels.

One solution could be a redesign of `expand_and_sum` with a separation of the leaking operations. However, since it is still possible that, at the micro-architectural level, information leaks, it may be interesting to look at countermeasures at the logical level. The following propositions aim either avoid using `expand_and_sum`, or make leakage of $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ not give away information on the key $\mathbf{y}$ or ensuring that the output of the Reed-Muller decoder cannot be exploited as in the Schamberger attack [43]. These changes counters our attacks on

$\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ as well as protecting against CCA-SCA such as [15,46,20]. In Section 7.5, we propose a discussion on the effectiveness of the SIMD as a countermeasure.

7.1 Reed-Muller decoder redesigned as minimum distance decoder

The Reed-Muller codes used in HQC are [384, 8, 192] and [640, 8, 320]. Because of their low dimension, the total number of codewords is $2^k = 256$. A first countermeasure would be to replace the current decoding with a simple minimum distance decoder. This would require the codeword list which takes respectively 12 288 and 20 480 bytes of storage. Minimum distance decoding consists in xoring the codewords with the input of the decoder (easily masked operations) and computing the weight of the distance, these operations being in $\mathcal{O}(n \cdot 2^k)$. It remains to compare each distance to determine the index of the correct codeword, which is the decoded word, and is done in $\mathcal{O}(2^k)$.

As an alternate implementation aiming to reduce the memory requirements of the list, it is possible to just store the generator matrix (k codewords) and compute the remaining codewords, which however is done in $\mathcal{O}(n \times 2^k)$.

These versions of the algorithm are highly parallelisable, and do not leak Hamming weight information on the decoder input during the xoring if as the result can be written over the decoder input (the distance is the Hamming weight of the codeword, which is known). This makes our attacks inapplicable. However, the output of the decoder is not masked and may leak information during writing. This countermeasure is therefore not enough to avoid Zero Tester attacks.

7.2 Adding noise

A second countermeasure consists to acknowledge the leak on $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$, but add noise to render useless the information retrieved by the attacker. Instead of getting $\mathbf{mG} + \mathbf{e}'$ from side-channel information, adding noise makes the attacker retrieve $(\mathbf{mG} + \mathbf{e}' + \mathbf{e}'')$, with $\mathbf{e}''$ of weight $w_{\mathbf{e}''}$ such that $\binom{n}{w_{\mathbf{e}''}} > 2^\lambda$. This way, the attacker cannot guess the positions of the errors, and when $\mathbf{u}$ is invertible, $\mathbf{u}^{-1} \cdot (\mathbf{v} - (\mathbf{v} - \mathbf{u} \cdot \mathbf{y} + \mathbf{e}''))$ is no longer equal to $\mathbf{y}$. With the same reasoning, we could not infer bits of $\mathbf{y}$ when $\mathbf{u}$ is not invertible.

In order for the noise bits positions to not be revealed during replay attacks, they have to be determined deterministically from the ciphertext. Moreover, in order to avoid chosen-ciphertext attacks, the noise should also be derived from a secret value only known from the receiver. This way, it also denies the oracle used by Schamberger et al. as they leak the error intentionally set to $\mathbf{y}$. For the current parameter sets, $w_{\mathbf{e}''}$ would have to be at least 11/16/21 for security category I. $\omega(\mathbf{e}')$ is on average $\frac{p^*}{n}$, where $p^* = 0.3398$, the average $\omega(\mathbf{e}')$ is 6003.9262. When adding an extra $w_{\mathbf{e}''}$ noise bits, on average $w_{\mathbf{e}''} \cdot p^* = 3.7378$ of them will land on ones of $\mathbf{e}'$ and remove noise. The actual noise added is on average $w_{\mathbf{e}''}(1 - 2p^*)$, which brings the final weight to an average of $\omega(\mathbf{e}' + \mathbf{e}'') = 6007.4506$, which then increases p^* to 0.3400.

Given the improved formulae for decoding failure rates provided in [2, Prop. 6, Th. 7], the $\log_2(DFR)$ went from -132.8557 to -132.2368. Given that the

DFR of the Reed-Muller code is an upper bound that still has margins from the experimental DFR, it can be expected that this change has a negligible impact on HQC. The same reasoning applies for the other security categories, for which precise values can be found in Table 4.

Cat.	n	p^*	$\overline{\omega(\mathbf{e}')} $	$w_{\mathbf{e}''}$	$\overline{\omega(\mathbf{e}' + \mathbf{e}'')} $	$p^{*'} $	$DFR(p^*) $	$DFR(p^{*'}) $
I	17669	0.3398	6003.927	11	6007.451	0.340	-132.856	-132.237
III	35851	0.3618	12970.892	16	12976.754	0.362	-193.878	-193.122
V	57637	0.3725	21469.783	21	21475.137	0.376	-260.512	-259.842

Table 4: Consequences of adding noise to hide $\mathbf{mG} + \mathbf{e}'$. The notation $\overline{\omega(\cdot)}$ represents the average of $\omega(\cdot)$. DFR are expressed as a power of 2.

7.3 Codeword masking

Another countermeasure could be to make use of codeword masking [30]: the receiver generates a new message $\mathbf{m}'$ and computes $\mathbf{v}' = \mathbf{v} + \mathbf{m}'\mathbf{G}$ before decoding. As such, the previously used equality $\mathbf{v} - (\mathbf{v} - \mathbf{u} \cdot \mathbf{y}) = \mathbf{u} \cdot \mathbf{y}$ does not hold anymore and the attacker cannot continue the attacks introduced in this paper. If this codeword mask is determined deterministically from the ciphertext, a CCA-SCA such as [15,46,20] can still exploit the output of the Reed-Muller decoder to gain information on $\mathbf{y}$. Therefore it has to be determined randomly and use it as a nonce, as it is common to do in side-channel masking techniques [8].

The unmasking can be done easily by computing $\mathbf{m} = \mathcal{C}.decode(\mathbf{v} - \mathbf{u} \cdot \mathbf{y} + \mathbf{m}'\mathbf{G}) - \mathbf{m}'$. The masking should still hold for the KEM check of good reception of the message. This has the advantage that, the masking still applies during the polynomial samplings of the Reed-Solomon decoder, which makes this method a useful way to avoid the cited timing attacks. The other positive aspect of this method is that it is not dependent on the choice of the code. If a more efficient code or code combination was to replace RMRS, the codeword masking would still protect $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ and other internal values from leaking.

7.4 Countermeasures effectiveness

All of the previously mentioned countermeasures were tested against a t-test. This was done on the reference HQC to match the available instructions of the known RISC-V simulator [32] with 30 samples. Unlike the version without any countermeasure, the one using a minimum distance decoder for Reed-Muller only has potential leaks in the Reed-Solomon decoder as expected. In comparison, the noisy and codeword masking countermeasures did not show any possible

leakage. The overhead caused by each of the proposed countermeasures during the decrypt operation is depicted in Table 5.

	Proposed countermeasures to prevent our attacks							
Min. dist. decoder	×	✓	×	×	✓	×	✓	✓
Adding noise	×	×	✓	×	✓	✓	×	✓
Codeword masking	×	×	×	✓	×	✓	✓	✓
Cycles ($\times 10^3$)	62875	66232	62963	63589	66320	63660	66929	67017
Overhead	0%	+5.34%	+0.14%	+1.14%	+5.48%	+1.25%	+6.45%	+6.59%

Table 5: Comparison of the overhead induced by each countermeasure

7.5 SIMD operations

One decisive advantage of error correcting codes (including HQC) regarding power side-channel attacks is the factorial complexity of finding precise bitflip indexes when only observing the bitflip count as the number of indexes grow. For instance, only 1 unknown bitflip index per 16-bit blocks in the error vector is sufficient to make the attack infeasible in practice. For HQC-128, with code lengths $N_1 = 46$ and $N_2 = 384$, it would require at worst $\log_2 \left(\binom{16}{8} \right) * N_1 N_2 / 16 = 4416$ bits to bruteforce. With `-O3` option, the compiler starts to generate SIMD instructions to compute sixteen AND operations en parallel, leading to infeasible exploitation of this operation. In that sense, SIMD operations are good candidates for ISA level hardening. However, this countermeasure is facing two structural weaknesses.

First, SIMD operations are associated with non-SIMD instructions to load data or compute additional arithmetic and logical operations. For instance, with the `-O2` option, only the shift operations are vectorized in the `expand_and_sum`, meaning that specific software hardening and security assessment are required to ensure that there is no remaining leakage.

Second, the design of a SIMD hardware module that do not expose bit value from the micro-architecture’s leakage is an open research problem due to the circuit complexity of this type of hardware accelerator. Notice that previously proposed countermeasures remain relevant in a SIMD setting, increasing the security level of SIMD implementations.

8 Conclusion

This paper presents two side-channel (power consumption) attacks in the current optimized implementation of the HQC cryptosystem, based on concatenated

Reed-Muller and Reed-Solomon codes. These attacks allow an adversary to recover the secret key on targets not implementing SIMD instructions.

Existing side-channel attacks choose specific ciphertexts in order to make the leakage of the output of the Reed-Muller decoder reveal information on the secret key. Most of these ciphertexts are invalid, making the attack detectable.

Our attacks aim at recovering from power consumption the input of the Reed-Muller decoder $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ by replaying the same decryption. Given this information and the ciphertext $(\mathbf{u}, \mathbf{v})$ it is possible to deduce the secret key $\mathbf{y}$. The specific routine leaking is `expand_and_sum` which collects in 16-bits blocks the individual segments of the duplicated Reed-Muller code.

The first attack only uses the Hamming Weight of the registers state changes at the Instruction Set Architecture level. It targets a loop of four leaking instructions over each bit of the 16-bit block: a load, a right-shift, an add and a store. The last loop gives direct information on the last bit. From this information, each previous bit can successively be deduced. We show the applicability of the attack on a RISC-V simulator, and show that the attack is also effective on most common ISA and optimisation levels that do not use SIMD instructions.

The second attack exploits a correlation between each bit and power consumption at specific POI. The micro-architecture leaks information that can be leveraged, and experiments were successfully conducted with a ChipWhisperer Pro over a Cortex-M4. Results show that the traces statistically separate each bit such that 83 traces gives a success rate of over 99%.

Finally three countermeasures at the algorithmic level are proposed to prevent the decryption from leaking information: redesigning the decoding, adding noise to the ciphertext and masking it with another random codeword. The consequences of these countermeasures are also discussed.

Acknowledgments

This work was partially supported by the Institut Cybersecrit  Occitanie (ICO) and the joint-lab SEIDO with the active participation of Arthur Villard, EDF R&D, France.

References

1. Aguilar Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Bos, J., Deneuville, J.C., Dion, A., Gaborit, P., Lacan, J., Persichetti, E., Robert, J.M., V ron, P., Z mor, G.: Hamming Quasi-Cyclic (HQC). NIST PQC Round 4(10), 52 (2024)
2. Aguilar Melchor, C., Aragon, N., Deneuville, J.C., Gaborit, P., Lacan, J., Z mor, G.: Efficient error-correcting codes for the HQC post-quantum cryptosystem. Designs, Codes and Cryptography 92(12), 4511–4530 (2024). <https://doi.org/10.1007/s10623-024-01507-6>
3. Aguilar-Melchor, C., Blazy, O., Deneuville, J.C., Gaborit, P., Z mor, G.: Efficient encryption from random quasi-cyclic codes. IEEE Transactions on Information Theory 64(5), 3927–3943 (2018)

4. Alekhnovich, M.: More on average case vs approximation complexity. In: 44th Annual Symposium on Foundations of Computer Science. pp. 298–307. IEEE Computer Society Press, Cambridge, MA, USA (Oct 11–14, 2003). <https://doi.org/10.1109/SFCS.2003.1238204>
5. ARM: Arm gnu toolchain downloads (2024), <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>
6. Baïsse, C., Moran, A., Goy, G., Maillard, J., Aragon, N., Gaborit, P., Lecomte, M., Loiseau, A.: Secret and shared keys recovery on hamming quasi-cyclic with sasca. *Designs, Codes and Cryptography* pp. 1–21 (2025)
7. Berlekamp, E., McEliece, R., Van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information theory* **24**(3), 384–386 (2003)
8. Casalino, L., Belleville, N., Couroussé, D., Heydemann, K.: A tale of resilience: On the practical security of masked software implementations. *IEEE Access* **11**, 84651–84669 (2023). <https://doi.org/10.1109/ACCESS.2023.3298436>
9. Cheng, S.H.W., Chuengsatiansup, C., Genkin, D., McNeil, D., Murray, T., Yarom, Y., Zhang, Z.: Evict+spec+time: Exploiting out-of-order execution to improve cache-timing attacks. *Cryptology ePrint Archive, Report 2024/149* (2024), <https://eprint.iacr.org/2024/149>
10. Couvreur, A., Gaborit, P., Gauthier-Umaña, V., Otmani, A., Tillich, J.P.: Distinguisher-based attacks on public-key cryptosystems using reed-solomon codes. *Designs, Codes and Cryptography* **73**(2), 641–666 (2014). <https://doi.org/10.1007/s10623-014-9967-z>
11. Faugere, J.C., Gauthier-Umana, V., Otmani, A., Perret, L., Tillich, J.P.: A distinguisher for high-rate mceliece cryptosystems. *IEEE Transactions on Information Theory* **59**(10), 6830–6844 (2013)
12. Faugère, J.C., Otmani, A., Perret, L., de Portzamparc, F., Tillich, J.P.: Folding alternant and goppa codes with non-trivial automorphism groups. *IEEE Transactions on Information Theory* **62**(1), 184–198 (2015)
13. Goodwill, G., Jun, B., Jaffe, J., Rohatgi, P.: A testing methodology for side-channel resistance validation, https://csrc.nist.gov/csrc/media/events/non-invasive-attack-testing-workshop/documents/08_goodwill.pdf
14. Goy, G., Maillard, J., Gaborit, P., Loiseau, A.: Single trace HQC shared key recovery with SASCA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(2), 64–87 (2024). <https://doi.org/10.46586/tches.v2024.i2.64-87>
15. Guo, Q., Hlauschek, C., Johansson, T., Lahr, N., Nilsson, A., Schröder, R.L.: Don’t reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2022**(3), 223–263 (2022). <https://doi.org/10.46586/tches.v2022.i3.223-263>
16. Guo, Q., Johansson, T., Löndahl, C.: A new algorithm for solving ring-lpn with a reducible polynomial. *IEEE Transactions on Information Theory* **61**(11), 6204–6212 (2015)
17. Guo, Q., Johansson, T., Stankovski, P.: A key recovery attack on MDPC with CCA security using decoding errors. In: Cheon, J.H., Takagi, T. (eds.) *Advances in Cryptology – ASIACRYPT 2016, Part I. Lecture Notes in Computer Science*, vol. 10031, pp. 789–815. Springer Berlin Heidelberg, Germany, Hanoi, Vietnam (Dec 4–8, 2016). https://doi.org/10.1007/978-3-662-53887-6_29
18. Guo, Q., Nabokov, D., Nilsson, A., Johansson, T.: SCA-LDPC: A code-based framework for key-recovery side-channel attacks on post-quantum encryption

- schemes. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023, Part IV*. Lecture Notes in Computer Science, vol. 14441, pp. 203–236. Springer, Singapore, Singapore, Guangzhou, China (Dec 4–8, 2023). https://doi.org/10.1007/978-981-99-8730-6_7
19. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) *TCC 2017: 15th Theory of Cryptography Conference, Part I*. Lecture Notes in Computer Science, vol. 10677, pp. 341–371. Springer, Cham, Switzerland, Baltimore, MD, USA (Nov 12–15, 2017). https://doi.org/10.1007/978-3-319-70500-2_12
 20. Huang, S., Sim, R.Q., Chuengsatiansup, C., Guo, Q., Johansson, T.: Cache-timing attack against HQC. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2023**(3), 136–163 (2023). <https://doi.org/10.46586/tches.v2023.i3.136-163>
 21. Ingleton, A.W.: The rank of circulant matrices. *Journal of the London Mathematical Society* **s1-31**(4), 445–460 (10 1956). <https://doi.org/10.1112/jlms/s1-31.4.445>, <https://doi.org/10.1112/jlms/s1-31.4.445>
 22. Kaplan, D.A.: Optimization and amplification of cache side channel signals (2023), <https://arxiv.org/abs/2303.00122>
 23. Katzman, D., Kosasih, W., Chuengsatiansup, C., Ronen, E., Yarom, Y.: The gates of time: Improving cache attacks with transient execution. In: Calandrino, J.A., Troncoso, C. (eds.) *USENIX Security 2023: 32nd USENIX Security Symposium*. pp. 1955–1972. USENIX Association, Anaheim, CA, USA (Aug 9–11, 2023)
 24. Kocher, P.C.: Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In: Koblitz, N. (ed.) *Advances in Cryptology – CRYPTO’96*. Lecture Notes in Computer Science, vol. 1109, pp. 104–113. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 18–22, 1996). https://doi.org/10.1007/3-540-68697-5_9
 25. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M.J. (ed.) *Advances in Cryptology – CRYPTO’99*. Lecture Notes in Computer Science, vol. 1666, pp. 388–397. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–19, 1999). https://doi.org/10.1007/3-540-48405-1_25
 26. Löndahl, C., Johansson, T., Koochak Shooshtari, M., Ahmadian-Attari, M., Aref, M.R.: Squaring attacks on mceliece public-key cryptosystems using quasi-cyclic codes of even dimension. *Designs, Codes and Cryptography* **80**, 359–377 (2016)
 27. Maillet, N., Nugier, C., Migliore, V.: Additional material (2025), <https://gitlab.laas.fr/nmaillet/kr-sca-hqc-additional-materials>
 28. McEliece, R.J.: A public-key cryptosystem based on algebraic coding theory. *Coding Theory* **4244**, 114–116 (1978)
 29. Melchor, C.A., Aragon, N., Deneuville, J., Gaborit, P., Lacan, J., Zémor, G.: Efficient error-correcting codes for the HQC post-quantum cryptosystem. *Des. Codes Cryptogr.* **92**(12), 4511–4530 (2024). <https://doi.org/10.1007/S10623-024-01507-6>, <https://doi.org/10.1007/s10623-024-01507-6>
 30. Merli, D., Stumpf, F., Sigl, G.: Protecting PUF error correction by codeword masking. *Cryptology ePrint Archive*, Report 2013/334 (2013), <https://eprint.iacr.org/2013/334>
 31. Migliore, V.: *Crypto riscv* (2024), <https://gitlab.laas.fr/vmiglior/imprise>
 32. Migliore, V., Maillet, N., Gharbi, S.: *Risc-v simulator* (2022), <https://gitlab.laas.fr/vmiglior/risc-v-simulator>
 33. NIST: Post-quantum cryptography | csrc (2017), <https://csrc.nist.gov/Projects/post-quantum-cryptography/faqs>

34. NIST: Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf> (dec 2016)
35. Nugier, C., Migliore, V.: Acceleration of Classic McEliece Post-Quantum Cryptosystem with Cache Processing. *IEEE Micro* **44**(1), 59 – 68 (Jan 2024). <https://doi.org/10.1109/MM.2023.3304425>, <https://laas.hal.science/hal-04232870>
36. Osvik, D.A., Shamir, A., Tromer, E.: Cache attacks and countermeasures: The case of AES. In: Pointcheval, D. (ed.) *Topics in Cryptology – CT-RSA 2006*. Lecture Notes in Computer Science, vol. 3860, pp. 1–20. Springer Berlin Heidelberg, Germany, San Jose, CA, USA (Feb 13–17, 2006). https://doi.org/10.1007/11605805_1
37. Percival, C.: Cache missing for fun and profit (08 2009)
38. Purnal, A., Bogner, M., Piessens, F., Verbauwheide, I.: ShowTime: Amplifying arbitrary CPU timing side channels. In: Liu, J.K., Xiang, Y., Nepal, S., Tsudik, G. (eds.) *ASIACCS 23: 18th ACM Symposium on Information, Computer and Communications Security*. pp. 205–217. ACM Press, Melbourne, VIC, Australia (Jul 10–14, 2023). <https://doi.org/10.1145/3579856.3590332>
39. Randriambololona, H.: The syzygy distinguisher. *Cryptology ePrint Archive*, Paper 2024/1193 (2024), <https://eprint.iacr.org/2024/1193>, to appear at Eurocrypt 2025
40. Rao, A., Bhimasankaram, P.: *Linear Algebra. Texts and Readings in Mathematics*, Hindustan Book Agency (2000), <https://books.google.fr/books?id=ZfJdDwAAQBAJ>
41. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM* **21**(2), 120–126 (1978)
42. Saab, S., Leiserson, A., Tunstall, M.: Efficient key extraction from the primary side of a switched-mode power supply. *Cryptology ePrint Archive*, Report 2015/512 (2015), <https://eprint.iacr.org/2015/512>
43. Schamberger, T., Holzbaur, L., Renner, J., Wachter-Zeh, A., Sigl, G.: A power side-channel attack on the reed-muller reed-solomon version of the HQC cryptosystem. In: Cheon, J.H., Johansson, T. (eds.) *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022*. pp. 327–352. Springer, Cham, Switzerland, Virtual Event (Sep 28–30, 2022). https://doi.org/10.1007/978-3-031-17234-2_16
44. Schamberger, T., Renner, J., Sigl, G., Wachter-Zeh, A.: A power side-channel attack on the cca2-secure hqc kem. In: Liardet, P.Y., Mentens, N. (eds.) *Smart Card Research and Advanced Applications*. pp. 119–134. Springer International Publishing, Cham (2021)
45. Schneider, T., Moradi, A.: Leakage assessment methodology - A clear roadmap for side-channel evaluations. In: Güneysu, T., Handschuh, H. (eds.) *Cryptographic Hardware and Embedded Systems – CHES 2015*. Lecture Notes in Computer Science, vol. 9293, pp. 495–513. Springer Berlin Heidelberg, Germany, Saint-Malo, France (Sep 13–16, 2015). https://doi.org/10.1007/978-3-662-48324-4_25
46. Schröder, R.L., Gast, S., Guo, Q.: Divide and surrender: Exploiting variable division instruction timing in HQC key recovery attacks. In: Balzarotti, D., Xu, W. (eds.) *USENIX Security 2024: 33rd USENIX Security Symposium*. USENIX Association, Philadelphia, PA, USA (Aug 14–16, 2024)
47. Sendrier, N.: Decoding one out of many. In: Yang, B.Y. (ed.) *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*. pp. 51–67. Springer Berlin Heidelberg, Germany, Tapei, Taiwan (Nov 29 – Dec 2 2011). https://doi.org/10.1007/978-3-642-25405-5_4

48. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM review **41**(2), 303–332 (1999)
49. Shoup, V., et al.: Ntl: A library for doing number theory
50. Yarom, Y., Falkner, K.: FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In: Fu, K., Jung, J. (eds.) USENIX Security 2014: 23rd USENIX Security Symposium. pp. 719–732. USENIX Association, San Diego, CA, USA (Aug 20–22, 2014)

A Methodology for the cycle count comparison

The statistics of table 1 were obtained from 100000 samples. All of those tests sources are available through the git instance [27].

To avoid Out-of-Order variations, each sample was obtained following these steps:

1. Flush the inputs of the function to benchmark from the caches
2. Memory fence
3. Read of TSC, the Time Stamp Counter, i.e. clock cycles since boot
4. Memory fence
5. Call to function to benchmark, i.e. the `expand_and_sum` implementation
6. Read of TSC

B Proof of Lemma 1

We prove Lemma 1 by case disjunction.

- In case $\mathbf{u} = 0$, $\mathbf{rot}(\mathbf{u})$ is the null matrix of order n and, as such, is of rank 0. Therefore, there is nothing to learn about $\mathbf{y}$ from $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ which is equal to $\mathbf{v}$.
- In the case where $\mathbf{u} = 1 + X + \dots + X^{n-1}$, $\mathbf{rot}(\mathbf{u})$ is a matrix of order n full of 1, and, as such, is of rank 1. This case, too, brings little information on $\mathbf{y}$ as $\mathbf{u} \cdot \mathbf{y}$ is either equal to 0 or to $\mathbb{1}$ depending of the parity of $\mathbf{y}$'s weight. In this case, the parity of $\mathbf{y}$'s weight is the only information that leaks.
- In the case where $\mathbf{u} \neq 0$ and $\omega(\mathbf{u})$ is even, the rank of the circulant matrix $\mathbf{rot}(\mathbf{u})$ can be computed in the general case as described in [21, Prop 1.1]: $\mathbf{rk}(\mathbf{rot}(\mathbf{u})) = n - d$ where d is the degree of $\gcd(\mathbf{u}, X^n - 1)$ and where $\mathbf{u}$ is viewed as a polynomial in $\mathbb{F}_2[X]$. Moreover, using the fact that n is a primitive prime for each version of HQC defined, the cyclotomic polynomial can be factored as: $X^n - 1 = (X - 1) \cdot (1 + X + \dots + X^{n-1})$. In this case $\omega(\mathbf{u})$ is even, so $\mathbf{u}(1)$ is a sum of an even number of 1: $\mathbf{u}(1) = 0$ as $\mathbb{F}_2$ is of characteristic 2. 1 being a root of $\mathbf{u}$, it is thus possible to factor $\mathbf{u}$ by $X - 1$. Moreover, $(1 + X + \dots + X^{n-1})$ being a monic irreducible polynomial, it is not possible to factor it by $\frac{\mathbf{u}}{X - 1}$. The degree of $(1 + X + \dots + X^{n-1})$ being superior to the one of $\frac{\mathbf{u}}{X - 1}$, it is also not possible to find it in the

decomposition of $\frac{\mathbf{u}}{X-1}$. As a result, $\gcd(\mathbf{u}, X^n - 1) = X - 1$ and is of degree 1. Using [21, Prop 1.1], it concludes the proof for $\mathbf{u} \neq 0$ and $\omega(\mathbf{u})$ even.

- In the case where $\mathbf{u} \neq 1 + X + \dots + X^{n-1}$ and $\omega(\mathbf{u})$ is odd, $\mathbf{u}(1)$ is a sum of an odd number of 1: $\mathbf{u}(1) = 1$ as it is computed in $\mathbb{F}_2$. However, 1 being the sole root of $X^n - 1$, it does not share any root with it. $\mathbf{u}$ also does not hold $1 + X + \dots + X^{n-1}$ as a divisor as it is of least degree per assumption. This means that $\gcd(\mathbf{u}, X^n - 1) = 1$, and per Ingleton [21, Prop 1.1], $\mathbf{rk}(\mathbf{rot}(\mathbf{u})) = n$.

This concludes the proof of Lemma 1. $\square$

C Signal-to-Noise Ratio

As experiments were performed on a low-noise target, a signal-to-noise ratio is given Figure 11 for the 10th bit.

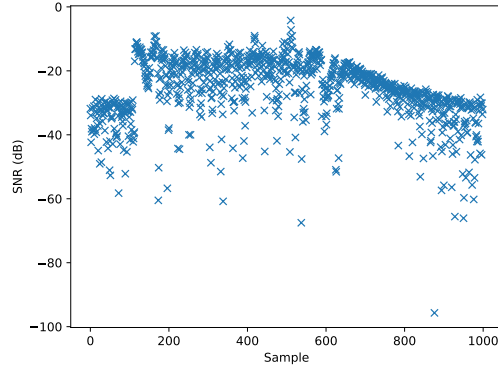


Fig. 11: SNR per sample for the bit_{10}